SynChat interview prep

Everything here is taken from the code, the commit history and docs/TECHNICAL.md. Answers are written the way you would say them. The stories in section 5 are reconstructed from what the repository records, so reword them to match how it went for you.

Contents

0. Read this first: your résumé describes a different SynChat
1. The pitch
2. Numbers to know cold
3. The path of one message
4. Technical questions by area
5. Behavioural questions and stories
6. The awkward questions
7. System design follow-ups
8. Concepts to explain from scratch
9. Before the interview

---

0. Read this first: your résumé describes a different SynChat

The résumé at `Portfolio/public/resume.pdf` (downloadable from arvin.life) says:

> Building a real-time chat app using Socket.IO, React (Vite), Express, and **MongoDB/Mongoose**. Implemented secure authentication with **JWT in HTTP-only cookies**, bcrypt hashing, and middleware for route protection.

That is the original MERN tutorial version. Today's SynChat is **PostgreSQL + pgvector** with **server-side sessions** (a random token in an httpOnly cookie, only its SHA-256 stored), not JWTs, and its headline feature (topic lanes) is not mentioned at all. An interviewer who reads the résumé and then the repository will ask about the mismatch. (The JWT-in-cookie design is actually what your SkillSwap project does.)

Fix the résumé before sending it. Suggested bullets, all defensible from this file:

- Built a messenger whose group chats sort themselves into topic lanes as messages arrive, using an on-server multilingual embedding model and a fitted scoring function: 58% of messages in the right lane on held-out chats vs 40% baseline, 86% with an opt-in LLM pass.
- Designed reliable messaging on PostgreSQL: idempotent sends with client-generated ids, gap-free ordering under row locks, and reconnect catch-up through a per-conversation revision counter; topic chip reaches other screens in a median 136 ms.
- Hybrid search (Postgres full-text + pgvector HNSW, reciprocal rank fusion), web push, PWA; 245 backend tests against a real Postgres and 6 Playwright walk-throughs.

If it comes up before you fix it: "That line describes the first version, a MERN tutorial app. I rebuilt it on Postgres with server-side sessions; let me show you why I moved away from JWTs" (then 4.9).

---

1. The pitch

One sentence

SynChat is a messenger whose group chats sort themselves into topics: it works out which conversation each message belongs to as it arrives, with no one having to start a thread.

Thirty seconds

In a busy group chat, several conversations run at once and their messages interleave: dinner plans, an assignment, a trip. Threads do not fix that in casual groups, because nobody starts them. SynChat assigns every group message to a topic lane as it arrives, on my own server, using a small embedding model and a scoring function over conversational signals. I measured it: on hand-labelled test chats it puts 58% of messages in the right lane against 40% for doing nothing, and 86% when a group opts in to a language model that re-sorts the chat whenever it goes quiet. Around that is a complete chat app: Postgres, Socket.IO, idempotent sends, reconnect catch-up, search by meaning, web push and a PWA.

Two minutes

Add these to the thirty-second version, in this order:

1. **Where it started.** A MERN tutorial chat app. I rebuilt the backend in TypeScript on Postgres, fixed its security problems (the socket trusted a `userId` in the query string, and one endpoint returned every user's email), then built the topic feature on top.
2. **How assignment works.** Sending never waits. The message is stored and delivered, then a per-conversation queue embeds it in a worker thread and a pure scoring function picks a lane. The topic chip appears on other screens a median of 136 ms later.
3. **The interesting finding.** Embedding similarity turned out to be weak evidence for chat messages, about 70% right against 42% chance, and larger models did no better. Most of the signal is conversational: whose turn it is, which question is still open, who was named.
4. **Honesty about the numbers.** 58% is a real gain and also a long way from "the lanes are right". I wrote down where it fails, kept two of those failures as expected-failure tests, and made corrections easy in the UI.
5. **The optional LLM layer.** Off by default per group, because it sends messages to an outside provider. Its output is treated as untrusted and validated in code.

What makes it more than a chat app

If asked "what is the hard part", say: **conversation disentanglement**. It is a studied research problem with a standard benchmark (Ubuntu IRC, Kummerfeld et al., ACL 2019), and I evaluated against it with the standard metrics.

---

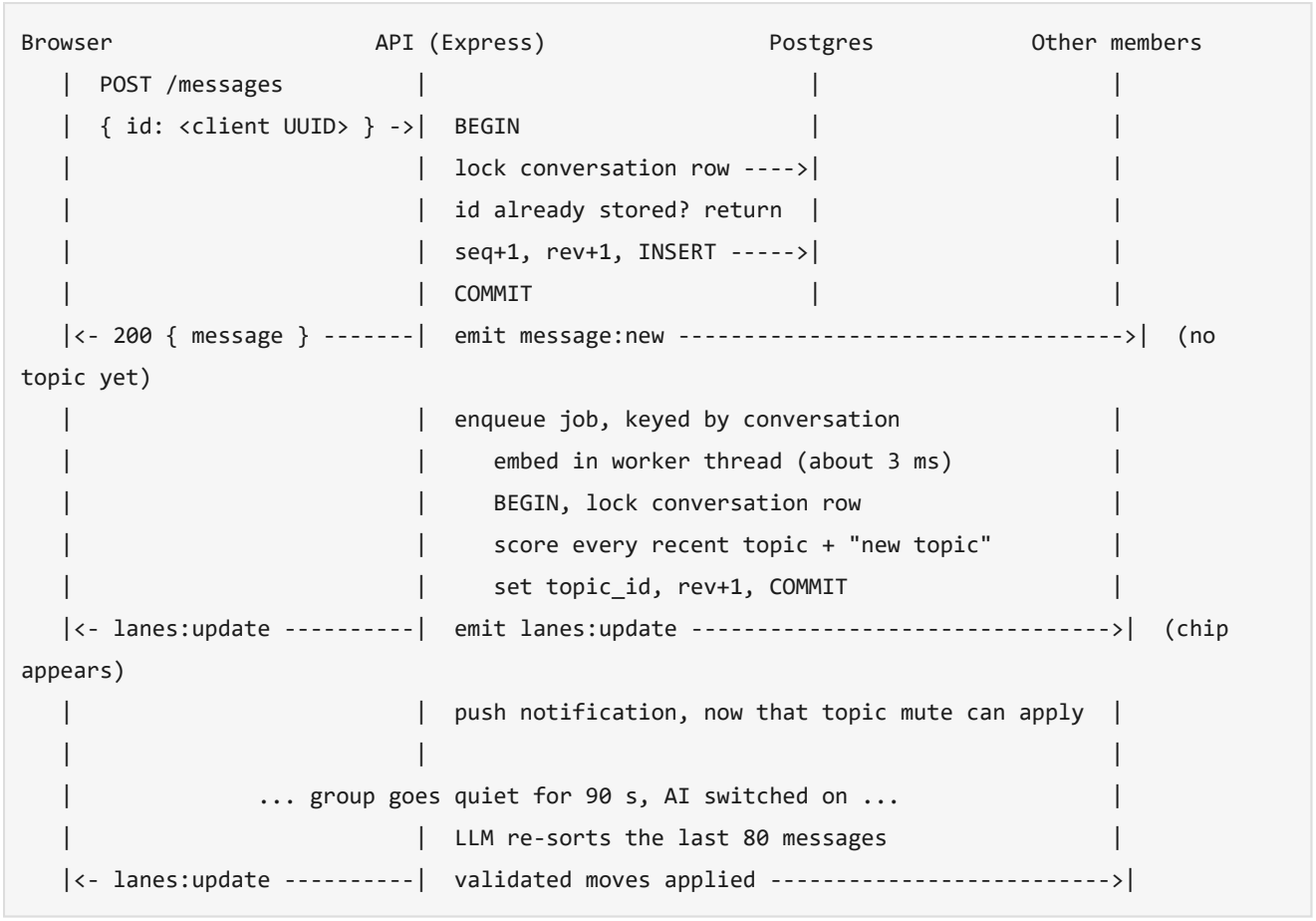
2. Numbers to know cold

What	Number	Caveat to say with it
Local assigner, held-out casual chats	**58.0%** one-to-one	5 chats, 207 messages, which I wrote
Baseline: whole chat as one lane	40.1%	
With the LLM tidy-up at each pause	**86.5%**	One run, `gemini-3.5-flash-lite`
One tidy-up after the last message	97.6%	Shows early decisions stick
Right choice given the true earlier lanes	81% held out	Drops to 58% when replayed, because mistakes compound
Ubuntu IRC test split, IRC-fitted weights	VI 91.5, one-to-one 78.3	Not the product's weights
Paper's feedforward model on the same split	VI 91.3, one-to-one 75.6	
Product weights on IRC	28.8 one-to-one	No better than the trivial baseline (27.6)
Time for the topic chip to appear on another screen	136 ms median	Three-browser walk-through, local machine
Embedding cost	about 3 ms per message on CPU	384 dimensions, quantised
Embedding alone picks the right lane	about 70%	Chance is 42%
Topic starts in casual chats	8% of messages	Missed 8 of 15 held out, invented 12
Lanes produced against true lanes	19 / 15	It over-splits
Evaluation data	20 chats: 15 tuning (603 messages), 5 held out (207)	About a third mixed Hindi and English
Backend tests	245 pass, 2 record known limitations	Run `npm test` and quote what it prints
Browser walk-throughs	6 scripts, 64 checks	Playwright
Assigner history window	200 messages	
LLM tidy-up window	80 messages, 500 characters each	
Quiet period before the LLM runs	90 seconds, capped at 10 minutes	

---

3. The path of one message

Be able to draw this on a whiteboard and talk through it.



Points to make while drawing:

- **The send never waits for the model.** Delivery and topic assignment are separate steps.
- **One conversation, one queue.** Assignments for one conversation run strictly in order. Different conversations run side by side.
- **The embedding happens before the lock is taken.** The slow part does not hold a row lock.
- **Everything that changes takes a `rev`.** A topic assignment is a change like an edit or a reaction, so it reaches reconnecting clients through the same catch-up path.

---

4. Technical questions by area

4.1 Messaging and reliability

How do you stop a retried send from creating a duplicate message? The client generates the message's UUID. On the server, the send runs in a transaction that first locks the conversation row with `SELECT ... FOR UPDATE`, then looks for that id. If it exists it returns the stored message; otherwise it inserts. Because the lock comes first, two concurrent retries queue up, and the second one finds the first one's row. There is a test that fires eight concurrent retries of one send and checks for exactly one row. (messages/service.ts)

Why lock the row instead of relying on the primary key and catching the unique violation? The lock also gives me ordering. The same statement that locks the conversation increments `last_seq` and `rev`, so both counters are gap-free and assigned in commit order. A unique-violation approach would handle

duplicates but not ordering. The unique violation is still caught, for the one case the lock cannot cover: the id already exists in a conversation this sender cannot see, which returns a 409.

What are `seq` and `rev`, and why two counters? `seq` orders messages within a conversation. `rev` counts every change: a new message, an edit, a delete, a reaction, a topic assignment. Each message is stamped with the `rev` of its latest change. A client that remembers the highest `rev` it has applied without a gap can ask `GET /conversations/:id/changes?since=<rev>` and get the current state of everything that changed. One mechanism covers new messages, edits and lane moves.

Why not use timestamps for ordering? Two messages can share a timestamp, clocks can go backwards, and a transaction's timestamp is set when it starts, not when it commits. A counter incremented under a row lock is in commit order by construction.

What happens when a client reconnects? On every socket `connect` event the client reloads the conversation list, then calls the changes endpoint for each open chat with its `syncedRev`. It also retries any sends that had failed. If too much changed, the server answers "reset" and the client reloads its latest page. (useChatStore.ts)

How does the client notice it missed a live event? It only advances `syncedRev` when an event's `rev` is exactly one more than the last. If a later `rev` arrives, something was skipped, so it schedules a sync 250 ms later, and skips that sync if the missing event has turned up by then. A stale copy of a message never overwrites a newer one, because the upsert compares `rev`.

How do optimistic sends work? The message goes into a `pending` list with its client-chosen id and is shown at once. The HTTP request runs in the background. On a network error, a 5xx or a 429 it retries with exponential backoff, capped at 15 seconds, up to 6 attempts, then is marked failed with a retry button. When the confirmed message arrives, by HTTP response or over the socket, the pending entry with the same id is dropped. For an image, the upload id is remembered so a retry does not upload the file again.

There is a subtle race between loading a page of history and syncing from it. How is it handled? `listMessages` reads the conversation's `rev` first and the messages second. Everything up to that `rev` is then guaranteed visible to the second query, so a client that later syncs from that `rev` cannot miss a change. Reading them the other way round could skip a change that committed in between.

How is delete handled? Delete for everyone erases the content and leaves a tombstone row, so `seq` stays gap-free and replies to it still resolve. The same transaction deletes the message's reactions, embedding and mentions, so nothing of a deleted message stays findable by keyword, meaning or mention.

4.2 Realtime

How are sockets authenticated? From the same httpOnly session cookie as the API, during the Socket.IO handshake. The client never tells the server who it is. The original tutorial code trusted a `userId` in the query string, so anyone could connect as anyone. (realtime.ts)

CORS covers this, surely? No. CORS does not apply to WebSocket handshakes. The browser sends the cookie on a cross-site WebSocket connection, so without a check any site could open an authenticated socket. I check the `Origin` header in `allowRequest`.

How do you deliver an event to a group? Each socket joins a room named for its user. To reach a conversation, the server reads the current member ids from the database and emits to those user rooms. Membership is read at emit time, so someone who was just removed stops receiving immediately, and there is no per-conversation room to keep in step with the database.

How does presence work with several devices? A count of sockets per user. Only the first socket announces "online", and going offline is announced after a 4-second grace period, which covers a page reload. Presence is only told to people who share a conversation with the user.

What happens on logout? Each socket also joins a room named for its session. Logging out deletes the session row and disconnects that session's sockets, so the other devices stay connected.

Why Socket.IO and not raw WebSockets or server-sent events? It was in the original app, and it gives reconnection, rooms and a fallback transport for free. Typing indicators need a client-to-server channel, which rules out server-sent events alone. If I were starting again with only server-to-client events I would consider SSE, because it goes through ordinary HTTP infrastructure.

4.3 Topic lanes: the assigner

Explain the assigner in one minute. It is a ranking model. For a new message, every recently active topic gets a score: a weighted sum of features describing how well the message fits. "Start a new topic" gets a score too, from features of the message alone. The highest score wins. It is a pure function: a message and the last 200 messages in, a decision out, with no database, model or clock inside. (assigner.ts)

What are the features? Six groups:

- **Meaning:** centred embedding similarity to what the topic has said, and shared distinctive words.
- **Recency:** how recently the topic was active, and what was said in the last few messages.
- **Author:** whether this person spoke here recently, and what share of their last six messages were here.
- **Turn-taking:** whether the last thing said in the topic was a reply to this person.
- **Questions:** whether the topic has an unanswered question and this message looks like an answer.
- **Addressing:** whether the message names someone who is taking part in the topic.

Why a linear model and not a neural network or an LLM for every message? Three reasons. I had 600 labelled messages, which is enough to fit about 35 weights and nowhere near enough to train a network. It has to run on every message on a CPU within milliseconds. And I can read it: when it puts a message in the wrong lane I can print each feature's contribution and see why, which is how the features were developed. An LLM per message would cost money per message and would send every group's text to a third party by default.

How do you handle "ok" and "lol"? Those carry no meaning of their own. I first estimate how informative a message is, from 0 for "ok" to 1 for a full sentence, by counting content words after removing filler. Every meaning feature is multiplied by that number. So for "ok" the embedding counts for nothing and the conversational features decide: whose turn it is, which question is waiting, what was just said. The filler list includes Hindi and Hinglish fillers, and stretched spellings like "nooo" are normalised first. (text.ts)

Are there cases with no scoring at all? Two. An explicit reply joins the topic of the message it replies to, and a message sent from inside a lane view is pinned to that lane at send time.

Why is it a pure function? So that the server and the offline evaluation run the same code with the same shapes. The accuracy numbers describe what runs in production, not a reimplementation of it. It also makes it trivial to unit test: each feature has a test.

Where do the weights come from? Fitted by maximum likelihood in eval/fit.ts. For each labelled message, I build the features from the true topics of the earlier messages, apply a softmax over the candidate scores, and maximise the probability of the true choice, with L2 regularisation. Then two adjustments made on replayed conversations: topic starts count double, because they are rare and expensive to miss, and the weights of content evidence and question-answer ties are scaled by 1.5, because the likelihood under-weights features that fire rarely.

How do you avoid two assignments racing each other? A keyed serial queue. Jobs with the same key run one at a time in order; jobs with different keys run concurrently. The key is the conversation id. It is 36 lines: a map from key to the tail of a promise chain. On top of that, the assignment transaction locks the conversation row, which also serialises it against moves and merges made by people. (queue.ts)

The queue is in memory. What if the server crashes? Pending jobs are lost, so at startup `recover()` finds messages from the last seven days that lack a topic or an embedding and queues them again. Jobs are written to be safe to run twice: the embedding insert uses `ON CONFLICT DO NOTHING`, and a message that already has a topic is not reassigned.

Why does the embedding run in a worker thread? Inference is CPU-bound. On the main thread it would block the event loop that serves every HTTP request and socket event. In a worker it runs in parallel and the result comes back as a transferred buffer, with no copy.

How are topic titles made without a model? From the opening message, minus greetings, mentions and links, cut at a word boundary. A topic that opened with "hey guys" is titled by the first message that says what it is about. A title a person chose is never replaced.

What happens when someone corrects a lane? Move, merge or rename applies for everyone, travels through the same `rev` mechanism, and is written to `topic_corrections` along with what the assigner had decided and its score. That is labelled data from real use, which is what the evaluation says the model most needs.

4.4 Embeddings and centring

Which model, and why? `paraphrase-multilingual-MiniLM-L12-v2`, 384 dimensions, quantised to 8 bits, run with `@huggingface/transformers`. It is small, multilingual, about 3 ms per message on CPU, and the text never leaves the server. I tested three larger models on the lane-picking task and none did better, so the small one is the right one.

What is "centring" and why did you need it? Sentence models place texts of the same language and register close together regardless of subject. For romanised Hindi this is extreme: any two Hinglish messages score 0.3 to 0.9, against 0.1 to 0.3 for English. No single threshold serves both. So before comparing two messages I subtract the mean vector of the conversation's recent messages from both. What is left is about 0 for unrelated messages in any language. (centre.ts)

What about a conversation with only three messages? The mean would be those messages. The mean is shrunk toward zero, as if five zero vectors were averaged in. With three messages the mean is divided by eight, so it has little effect; with 200 it is nearly the true mean.

Where is that computed? In SQL, with pgvector. One query takes the last 200 messages, averages their vectors, subtracts the mean from both sides and returns the cosine similarity for each row, so no vectors travel back to Node. The evaluation computes the same thing in TypeScript, and a test checks the two agree to four decimal places. (lanes/service.ts)

What if the model fails to load? Lanes still work from the conversational signals alone, and search falls back to keywords and says so. The embedder is an interface, and the service accepts `null`.

4.5 Evaluation

How did you measure accuracy? Two datasets. The Ubuntu IRC disentanglement corpus, which is the standard benchmark, and 20 casual group chats I wrote and labelled, 15 for tuning and 5 held out. The metrics are the standard ones from the paper: VI, one-to-one, and exact-match precision, recall and F1.

What is one-to-one accuracy? Find the best one-to-one pairing between predicted lanes and true conversations, then count the percentage of messages that land in a matched pair. If I predict lanes A and B and the truth is X and Y, it pairs each of mine with at most one true conversation so as to maximise overlap.

What is VI? Variation of information: an information-theoretic distance between two clusterings. It is zero when they are identical. The reported number is rescaled so that 100 is perfect.

How do you know your metric code is right? The "link every message to the previous one" baseline reproduces the paper's published figures exactly: VI 66.1, one-to-one 27.6. The metrics are also tested against hand-computed values and a brute-force implementation.

Your VI is the same as doing nothing. Explain. Yes: 80.0 against 79.9 held out. VI is lenient toward lumping everything into one lane when the chat has few topics. The gain is in one-to-one accuracy only, and only 6% of lanes come out exactly right. I state that in the documentation rather than quoting the flattering metric.

You say 81% of decisions are right but only 58% end up in the right lane. Why the gap? Error compounds. The 81% is measured with the true earlier lanes given. In a real replay the model lives with its own earlier decisions, so one early wrong pick leaves a lane mixed and every later message builds on it. I measured where the loss comes from by replaying with parts of the decision made perfect: perfect topic starts alone give 71%, and a perfect pick among existing lanes alone gives 80%.

Why do the IRC weights fail on casual chats and the reverse? They are different problems. A crowded public help channel has dozens of speakers who address each other by nick, and most new voices start a conversation. Five friends rarely name each other and switch subject with no marker. I tried fitting on both together and using IRC weights as a prior; both were worse than fitting on casual chats alone.

What is wrong with your evaluation? Have this list ready, because volunteering it is stronger than being led to it:

- The casual set is small, synthetic, and written by the same person who wrote the assigner.

- The held-out chats were scored in aggregate while I iterated, so they are lightly used, not untouched.
- A second annotator would draw some topic boundaries differently, and I have no inter-annotator agreement figure.
- One run, with no significance testing.
- Nothing measures a chat longer than the 80-message LLM window.

What would a better evaluation look like? Real chats from consenting groups, labelled by two people with agreement measured, a test set that is scored once, and confidence intervals from bootstrapping over chats. The product already collects the first ingredient: every correction is logged with the assigner's original decision.

4.6 The LLM features

What does the LLM do? Two kinds of request. The lane tidy-up re-sorts the last 80 messages when a group goes quiet. The topic digest is one structured request per topic that three features read from: a two-line catch-up, a decided-or-open status, and an inbox of questions waiting on you.

Why only when the group goes quiet, and not per message? Cost, rate limits and quality. One request per quiet spell is cheap and predictable. And the model does much better with a whole exchange in front of it than with one message at a time.

How do you defend against prompt injection? On the way in: the system prompt says messages are data and never instructions, each message sits in its own tag, and its text is escaped so it cannot close the tag and pose as structure. But I do not rely on that. On the way out the answer is treated as untrusted: the model refers to messages, lanes and people by short references like `m7`, `L2`, `P3`, and anything that names a reference it was not given is dropped. The request has no tools and no conversation state, so the worst a successful injection can do is mis-sort 80 messages in one group.

Which rules are enforced in code? Three, in tidy.ts: a message a person placed is never moved, an explicit reply stays with the message it replies to, and nothing outside the 80 messages shown is touched. Also, if an admin switches AI off or someone moves a message while the model is working, that part of the answer is discarded. The transaction re-reads `ai_enabled` under a lock before applying anything.

What privacy decisions did you make? The features are off in every group until an admin turns them on. The switch names the provider and says in plain words that messages will be sent to it, and flipping it is announced in the chat. The model sees first names and short references only: no ids, usernames or emails. Turning it off deletes what the model produced. Without an API key the features do not exist: hidden in the UI, 503 from the API.

How do you support two providers? One small interface, `StructuredLlm`: a system prompt, one user turn, and an answer constrained to a JSON schema. Claude and Gemini are each one class behind it. The answer is validated with zod on my side regardless of what the provider promises. Provider errors, refusals and cut-off answers all become one `LlmUnavailable` error; nothing changes and the work is retried at the next quiet spell. (llm.ts)

What did you learn from building the tidy-up? The first version showed the model the local assigner's lane for every message and asked it to fix mistakes. That scored 74% held out. The same model sorting

from the text alone scored 85%. Being handed a wrong guess made it worse; it anchored on it. So I rebuilt it to hide the guess for messages not yet sorted.

The tidy-up moves messages between lanes. How do lane ids stay stable? When the model declares a "new" lane that mostly matches a topic the local assigner had already opened, the plan keeps that existing topic and its id and just retitles it, instead of deleting one topic and creating another. Clients that have that lane open are not disturbed.

4.7 Search

How does search work? Two searches over the caller's own conversations, merged. Keyword search uses Postgres full-text with a GIN index. Meaning search takes the nearest neighbours of the query's embedding through a pgvector HNSW index. The two rankings are combined with reciprocal rank fusion and the results grouped by topic. (search/service.ts)

Why both? Each finds what the other misses. "Where are we going to eat this weekend" shares no word with "who's in for dinner at the ramen place on saturday", which meaning search returns first. A name or a number has no meaning to embed, so keyword search finds those.

What is reciprocal rank fusion and why use it? Each result scores 1 / (60 + its rank) in each list it appears in, and the scores are added. It uses only ranks, so I never have to make a full-text rank and a cosine similarity comparable. 60 is the constant from the original paper.

Why the `simple` text search configuration? The English configuration stems words and drops stop words, which would mangle Hindi written in Latin letters. `simple` does neither. I drop filler words from the query myself, and prefix-match each word, so "book" finds "booked".

What is the problem with filtering an approximate nearest-neighbour search by membership? An HNSW index returns the nearest vectors overall, and the filter is applied afterwards. If the nearest 40 are all in other people's conversations, the caller gets nothing back, even though they have relevant messages. I set `hnsw.iterative_scan = 'relaxed_order'`, which makes pgvector keep scanning until enough rows pass the filter.

How do you make sure search never leaks another person's messages? Both candidate queries join `conversation_members` for the caller, and so does the final query that loads the rows that leave the server. There are tests for a non-member and for someone who was just removed, by keyword and by meaning.

4.8 Database

Why did you move from MongoDB to Postgres? Four things I needed that Postgres does in one store: transactions with row locks for the ordering guarantee, joins for the membership check on every query, full-text search, and vectors through pgvector. Keeping messages, text search and vectors together means a single query can filter by membership and rank by similarity.

Why raw SQL and no ORM? The SQL that enforces membership is visible at every call site, and the queries that matter here are ones an ORM does not express well: `FOR UPDATE OF`, lateral joins, vector arithmetic, generated `tsvector` columns. Migrations are numbered SQL files applied at startup.

Walk me through the schema. `users`, `sessions`, `conversations` with `last_seq` and `rev`, `conversation_members` with `role` and `last_read_seq`, `messages` with a client-generated primary key and `UNIQUE (conversation_id, seq)`, `topics`, `message_embeddings` with a `vector(384)` column, `topic_corrections`, `topic_mutes`, `message_mentions`, `push_subscriptions`.

How do you guarantee two people have only one DM? A `dm_key` column holding the two user ids in sorted order, with a unique constraint. A check constraint ties it to the type: a row is a DM if and only if it has a `dm_key`.

How are unread counts stored? Not as a counter. Each member has a `last_read_seq`, and unread is the number of messages with a higher `seq` from someone else. A topic's size, last activity and unread count are derived from its messages through an index on `(conversation_id, topic_id, seq)`, so they cannot drift after a move or a merge.

Which indexes, and why? `(conversation_id, seq)` for history pages. `(conversation_id, rev)` for reconnect catch-up. `(conversation_id, topic_id, seq)` for lane views and topic stats. `conversation_members (user_id)` for "which conversations am I in". GIN on the `tsvector` column. HNSW with cosine distance on the embeddings.

What is the downside of locking the conversation row on every send? All writes to one conversation are serialised. That is fine for a group chat, where a handful of people type at human speed and the transaction is short. It would be the bottleneck for a channel with thousands of concurrent senders, where I would move to a per-conversation sequence allocator or accept gaps.

4.9 Security

How does authentication work? Server-side sessions. Login creates a random 32-byte token, sets it in an httpOnly, SameSite=Lax cookie (Secure in production), and stores only its SHA-256 hash. Expiry slides, and the row is touched at most once an hour to avoid a write per request. (sessions.ts)

Why sessions and not JWTs? Revocation. Logging out deletes the row and the token is dead immediately, on every server. A JWT stays valid until it expires unless I add a server-side denylist, at which point I have sessions again with extra steps. The cost is one indexed lookup per request.

Why hash the session token if it is already random? So that a leaked database does not hand out usable sessions. SHA-256 is enough here because the token has 256 bits of entropy; a slow hash like bcrypt is for low-entropy secrets such as passwords.

How do you prevent CSRF? Two layers. The cookie is SameSite=Lax, so browsers do not send it on cross-site POSTs. And a middleware refuses any state-changing request whose `Origin` header is not one of mine. A request with no `Origin` is allowed, because a non-browser client has no ambient cookie to abuse.

How do you handle authorisation? One gate, `requireMember`, or a query that joins `conversation_members` for the caller. A non-member gets 404, the same answer as for an id that does not exist, so the API does not reveal which ids are real. This is tested for every route, over HTTP and over the socket, and for members who were just removed.

What about password handling? bcrypt at cost 12, a 10-character minimum, common passwords refused. Input longer than 72 bytes is refused rather than silently truncated, because bcrypt only reads the

first 72. On login with an unknown account I still run a comparison against a dummy hash, so response time does not reveal whether the account exists.

Rate limiting? Login is limited per IP and per (IP, account) pair. Keying the account limit on both means an attacker cannot lock a victim out from somewhere else. Sends, uploads, search, lookups and AI requests are limited per user.

Uploads? Multipart, capped at 5 MB, typed from the file's bytes rather than its name, no SVG, stored under random names. A message refers to an upload by id, so it cannot carry an arbitrary URL.

Why no end-to-end encryption? It is incompatible with the product. Topic assignment and search run on the server and need the text. I say so in the documentation as a deliberate decision.

What security weaknesses remain? Signup reveals whether an email is registered, because there is no email verification step to hide it behind. Uploaded images are served from unguessable URLs without a session check. There is no blocking or reporting. Rate-limit counters are in process memory.

4.10 Frontend

How is state managed? Zustand, with separate stores for auth, chat, config, per-user feature switches and theme. The chat store keeps, per conversation, a gap-free run of messages ending at the newest, a pending list, and the `syncedRev` cursor. Socket handlers call store actions directly.

Why Zustand over Redux or React context? Much less code for the same result, and components subscribe to slices, so a new message in one chat does not re-render the others. It was also in the original app. Socket event handlers live outside React, and Zustand's `getState()` makes that straightforward.

How does the message list handle long histories? `react-virtuoso` virtualises it, so only the visible rows are in the DOM, and older pages load as you scroll up. One of the browser walk-throughs loads a 300-message history.

What does the service worker do? It is hand-written, about 90 lines. Page navigations are network-first with a cached copy as the offline fallback, so a new deploy is picked up. Hashed assets are cache-first, since a hashed filename never goes stale. It never caches the API, the socket or uploads. It also shows push notifications and, on click, focuses an existing window instead of opening a second one. (sw.js)

Why hand-written and not Workbox? The whole policy is three rules, and I wanted to understand exactly what was cached. A stale cached API response in a chat app is a correctness bug.

How do push notifications respect topic mute? A group message's topic is only known a moment after it arrives, so a push sent at send time could not know whether the topic was muted. Notifications for group messages are sent from the lane job, once the topic is settled. One query selects the recipients: current members, not the sender, not anyone who muted that topic unless the message mentions them. People who are looking at the app right now are skipped, because they get it live.

How do you know someone is looking? The client reports window focus and visibility over the socket. Being connected is not enough: a background tab is connected and should still get a notification.

4.11 Testing

What is your testing approach? Backend tests run against a real Postgres, rebuilt from the migrations each run, with Vitest and Supertest. I did not mock the database, because most of the logic that matters is in the SQL and the locking. Then six Playwright walk-throughs, each driving two or three separate browser sessions through a build phase.

What do you test, specifically? Coverage follows the risks: the membership gate on every route, ordering under concurrent sends, racing idempotent retries, reconnect catch-up, multi-device presence, each assigner feature, the metrics against hand-computed values, hostile model output, and the exact shape of the request sent to the LLM.

What are the "known limitation" tests? Two cases the assigner gets wrong, written as `it.fails`: a change of subject in the middle of a fast exchange, and a short follow-up that goes to whichever lane spoke last. They pass while the assigner fails. If someone fixes the behaviour, the test breaks and tells them to promote it to a normal test.

What did the end-to-end tests catch that unit tests did not? Four real bugs: the typing indicator being dropped right after a send, topic mute being ignored for chats whose topics were not loaded, catch-up never appearing because opening a chat marks it read, and the embedding worker failing to load in the production build.

What is not tested? The real Anthropic API call, as no key was available. Cloudinary uploads. Push delivery to a real browser, because automated browser contexts have no Push API. Mobile browsers and screen readers.

4.12 Deployment

How is it deployed? In two halves. The web app is a static Vite build on Vercel. The API is a long-running Node process on a separate host, because it holds WebSocket connections, runs the embedding model in a worker thread and needs Postgres with pgvector, none of which fits serverless functions.

Why does the API need to be on a subdomain of the same site? The session cookie is SameSite=Lax, so the browser only sends it when the web app and the API share a registrable domain: `arvin.life` and `api.arvin.life`.

How are migrations run? Numbered SQL files applied automatically at startup, which `AUTO_MIGRATE=false` turns off.

What does `vercel.json` do? Builds `frontend/`, rewrites every route to `index.html` for client-side routing, serves `sw.js` with `no-cache` so a new service worker is picked up, and marks hashed assets immutable for a year.

What does `render.yaml` do? It is a Render Blueprint for the API: Node 24, `npm ci --include=dev` (so the TypeScript compiler is installed even with `NODE_ENV=production`), build, then `npm run model:fetch` to download the embedding model **at build time**. Downloading and loading the model at startup together needed more than the free plan's 512 MB. Health check on `/api/health`, `TRUST_PROXY=1` so rate limits see the real client IP behind Render's proxy, and every secret marked `sync: false` so it is typed into the dashboard, never committed.

The free plan sleeps. What does that mean for users? After 15 minutes without traffic the first request takes up to a minute while the server wakes. The README says so next to the demo link. On a paid plan or any always-on host the problem disappears; nothing in the code depends on it.

Where is the database? Any Postgres with pgvector; the README suggests Neon. Migrations enable the `vector`, `citext` and `pgcrypto` extensions themselves.

4.13 Chat features: how each one actually works

Interviewers who open the app will poke at the visible features. Know each one from the button to the table.

@mentions, end to end.

1. *Typing.* In MessageInput.tsx, every render runs the regex `/(?:^|\s)@([a-z0-9_.]*)$/i` on the text. If the text ends in `@` plus a partial name, the captured prefix filters the conversation's members (by username or by full name, excluding me) to at most five suggestions. Enter or Tab picks the first; clicking picks any. Picking replaces the trailing `@partial` with `@username `. Nothing is sent to the server while typing: the member list is already in the client's conversation object.
2. *Sending.* The message goes as plain text. The client does **not** send a list of mentioned ids, so a client cannot claim to have mentioned someone it did not write.
3. *Server parsing.* mentions.ts runs `(?:^|[^\w@.])@([a-z0-9_.]{3,24})` over the text. The leading group stops `email@domain.com` from counting as a mention of `domain.com`. Trailing dots are stripped ("thanks @asha.").
4. *Resolution.* One `INSERT ... SELECT` writes `message_mentions` rows for usernames that are **members of this conversation**, never the sender, `ON CONFLICT DO NOTHING`. Because only members can be mentioned, typing `@someone` never reveals whether that username exists elsewhere on the server.
5. *Same transaction.* This runs inside the send transaction, and again on edit (delete old mention rows, re-parse), and the mention rows are deleted when the message is deleted.
6. *Rendering.* The message DTO carries `mentions: string[]` (user ids, from a subquery in `MESSAGE_COLUMNS`). MessageText.tsx splits the text on `@name` tokens and bolds only tokens that match a resolved member; a mention of me gets a highlight. Everything is rendered as React text nodes, never `dangerouslySetInnerHTML`, so a message cannot inject markup.
7. *What mentions change.* A mention gets through a muted topic: the push query has `OR EXISTS (SELECT 1 FROM message_mentions ...)`, and the client's unread logic treats a mention the same way.

Reactions.

- Table `reactions (message_id, user_id, emoji)` with that triple as the primary key, so a person can add the same emoji only once but can add several different emojis.
- `PUT /api/messages/:id/reactions {emoji}` adds, `DELETE` with the same body removes. The client decides which by checking whether my id is already in that emoji's `userIds` (useChatStore.ts).
- The emoji is validated with a Unicode property regex (`\p{Extended_Pictographic}`, regional indicators for flags, modifiers and zero-width joiners for skin tones and family emojis), max 32 characters. So nobody can "react" with arbitrary text.
- At most 20 *different* emojis per message, to stop a message being turned into a wall.

- A reaction is a change like an edit: it goes through `mutate()`, which locks the conversation row, applies the change, and only if a row actually changed takes the next `rev` and stamps the message with it. Then `message:updated` goes to every member. Reconnecting clients get it through `/changes`, same as any edit.
- The DTO groups reactions per emoji with the list of user ids, ordered by who reacted first, built in SQL with `json_agg`. The UI shows emoji + count, highlighted if I am in the list, and the tooltip lists names.
- Not optimistic: the UI updates when the response comes back (the request is small and quick). If asked "why not optimistic?", say it is an easy addition but the toggle state depends on the server's view (someone else may have removed theirs), and the response carries the authoritative list.

Replies (quotes). The send carries `replyToId`. The server checks the target is in the same conversation. `MESSAGE_COLUMNS` joins the replied-to message and returns a preview: sender, first 160 characters, whether it had an image, whether it is deleted. Clicking the quote jumps to the original and highlights it. For lanes, an explicit reply always joins the topic of the message it replies to (`topic_source = 'reply'`).

Edit. Only your own, live, non-system messages. `UPDATE ... WHERE text IS DISTINCT FROM $2` so a no-op edit does not spend a `rev`. Sets `edited_at`, re-parses mentions, and tells the lane service (`messageEdited`), which deletes and recomputes the embedding on the conversation's queue so search by meaning matches the new text. The topic is not reassigned.

Delete for everyone. Your own message, or any message if you are a group admin. Text and image are set to NULL, reactions, embedding and mentions are deleted, and the row stays as a tombstone so `seq` has no gap and replies to it still render ("Message deleted").

Read receipts and unread counts. Each membership stores `last_read_seq`. When the chat is open and the window is actually visible and focused, the client posts `POST /conversations/:id/read {seq}`; the server only ever moves it forward (the `UPDATE` has `WHERE last_read_seq < LEAST($seq, c.last_seq)`, so it also cannot be set past the newest message) and broadcasts a `read` event only if it actually moved. My own messages show one tick, or two when every other member's `last_read_seq` is at least that message's `seq` ("Read by 2 of 3" in a group). Unread count = messages with a higher `seq` from someone else, computed in SQL, so it cannot drift. Sending a message also marks everything up to it as read for the sender.

Typing indicator. The client emits `typing` at most once every two seconds per conversation, and immediately after a send so the next message is announced at once. The server drops anything that is not a UUID, allows a burst of 4 per 2-second window per socket, checks membership, and forwards to the other members. The receiver shows it for a few seconds and clears it early when that person's message arrives.

Presence (online dot). See 4.2: a per-user socket count, a 4-second grace on going offline, and announcements only to people who share a conversation. On connect the client asks `presence:sync` with an acknowledgement callback to get the current online list.

Images. The client checks type and size (5 MB) and measures the image with `img.decode()` to get its width and height, so the list can reserve the right space before the image loads (no layout jump). Upload is `POST /api/uploads/image` (multipart, multer in memory, 1 file). The server sniffs the magic bytes (JPEG `FF D8 FF`, PNG signature, `GIF87a/89a`, `RIFF....WEBP`) and rejects anything else, including SVG because SVG can carry script. Storage is Cloudinary in production or local disk in development, under a random

UUID name. The upload returns an id; the message refers to it with `imageId`, and the server only accepts an upload that belongs to the sender. A retry after a failed send reuses the upload id instead of uploading again. Paste-to-attach works too.

Direct messages. `POST /conversations/dm` with a username, an invite code, or a user id. A raw user id is only accepted for someone you already share a conversation with, so ids cannot be used to reach strangers. `dm_key` (the two ids in sorted order, unique) guarantees one DM per pair, even if both people click "message" at the same moment.

Groups, roles and invites. The creator is admin. Admins can rename, change the photo, add by username, remove, promote and demote, create and revoke invite links, and switch AI on or off. Anyone can leave. Rules enforced in transactions: a group always keeps at least one admin (demoting the last admin is refused; if the last admin leaves, the longest-standing member is promoted; if the last member leaves, the group is deleted); every change writes a `system` message ("Asha made Ben an admin") so the history explains itself. Group invite links are 18 random bytes (base64url), expire after 7 days, can be revoked, count their uses, and the preview page reveals only the group's name, photo and size. Each user also has a personal invite code for DMs, which they can rotate to kill a leaked link.

Topic mute. Personal: a row in `topic_mutes`. It hides that topic's unread count and stops its push notifications, except when you are mentioned. Other devices follow through `topic:mute` and `conversation:updated` events.

Per-user smart switches. `users.smart` is a sparse JSON object of features the person turned off (missing key means on): lanes, topic mute, catch-up, status, inbox. The update is one SQL statement, `smart = (smart || off) - on`, so two switches flipped at the same moment cannot overwrite each other.

4.14 Authentication end to end: what happens to a password

Be able to narrate this without looking.

Sign up.

1. The form in SignUpPage.tsx posts `{ email, username, fullName, password }` as JSON with axios (`withCredentials: true` so cookies are sent and accepted cross-subdomain).
2. **The password travels in plain text inside the JSON body, protected by HTTPS (TLS).** It is never put in a URL or query string (those end up in logs and browser history), never stored in `localStorage`, and never logged.
3. The server validates with zod: email normalised to lower case, username `^[a-z0-9_.]{3,24}$`, password at least 10 characters, at most **72 bytes** (bcrypt ignores everything after 72 bytes, so longer input is refused instead of silently truncated), not in a small common-password list, and not containing the username.
4. `bcrypt.hash(password, 12)`: bcrypt generates a random 16-byte salt and runs 2^12 rounds of its key schedule. The stored string `$2b$12$<salt><hash>` contains the algorithm, the cost and the salt, so `compare` needs nothing else.
5. Insert the user. Duplicate username or email is caught as a unique-constraint violation (not a check-then-insert, which would race) and returned as a 409 with the field name.
6. A session is created and the cookie set (below). Response: `201 { user }` with no hash in it (`toMe` picks safe fields only).

Log in.

1. `{ identifier, password }`, where identifier is an email or a username.
2. Rate limits: 30 per 15 minutes per IP, and 8 *failed* attempts per 15 minutes per (IP, identifier) pair. Successful logins do not count.
3. Look the user up. Always run `bcrypt.compare`, against a precomputed dummy hash if there is no such user, so the response time does not reveal whether the account exists. Same 401 message for both cases.
4. Create a session, set the cookie.

The session and the cookie.

- Token: `randomBytes(32)` as base64url, 256 bits of randomness.
- Database stores only `sha256(token)` in `sessions.token_hash`, plus user agent, created, last seen, expires (30 days by default).
- Cookie `sid`: `httpOnly` (JavaScript cannot read it, so XSS cannot steal it), `SameSite=Lax` (not sent on cross-site POSTs), `Secure` in production (HTTPS only), `path=/`, `maxAge` = session TTL.
- Every request: `requireAuth` reads the cookie, hashes it, and joins `sessions` to `users` where not expired. Expiry slides, and the row is touched at most once an hour.
- The browser holds no token the app can see. The frontend never stores anything auth-related in `localStorage`; it calls `GET /api/auth/me` on load to find out who it is.
- Any 401 from a non-auth endpoint (an axios response interceptor) means the session ended elsewhere, so the client signs out locally. A network error does not sign you out.
- Logout deletes the session row, disconnects that session's sockets, deletes push subscriptions tied to that session (`ON DELETE CASCADE`), and clears the cookie. Other devices stay signed in.

Why not hash the password in the browser before sending it? Then the hash *becomes* the password: anyone who steals it can replay it. TLS already protects it in transit, and the server must hash it with a salt and a slow function anyway. Client-side hashing adds complexity without protecting against the threats that matter.

Why bcrypt and not SHA-256 for passwords? Passwords have low entropy, so an attacker with the database would guess them. SHA-256 is designed to be fast (billions per second on a GPU). bcrypt is deliberately slow and salted, and its cost factor can be raised as hardware gets faster. Argon2id is the more modern choice; bcrypt is fine and widely supported. The production config refuses a cost below 12.

Is there Google sign-in? No, SynChat is email or username plus password only. If asked why: it was not needed to demonstrate the product, and the session design does not care how you proved who you are, so adding OAuth would be a new route that ends in the same `createSession` call. (SafetyWatch, my other project, does have Google OAuth through Better Auth, so you can talk about how it works there.)

What is missing? Email verification, password reset, and changing your password. Signup reveals whether an email is registered. Say these plainly.

4.15 The full data schema

Twelve tables over seven migrations (backend/migrations/). Know the columns that carry a design decision.

Table	Key columns	Why it looks like this
`users`	`id uuid`, `email citext UNIQUE`, `username citext UNIQUE`, `full_name`, `password_hash`, `avatar_url`, `invite_code UNIQUE`, `smart jsonb`	`citext` makes uniqueness and lookup case-insensitive without `lower()` everywhere. `invite_code` is rotatable. `smart` stores only switched-off features.
`sessions`	`id`, `user_id → users CASCADE`, `token_hash bytea UNIQUE`, `user_agent`, `last_seen_at`, `expires_at`	Only the hash of the token. Indexed on `user_id` and `expires_at`.
`conversations`	`id`, `type ('dm'/'group')`, `name`, `avatar_url`, `created_by`, `dm_key UNIQUE`, `last_seq`, `rev`, `last_message_at`, `ai_enabled`, `ai_enabled_by`, `ai_enabled_at`, `tidy_through_seq`	CHECK: is a DM iff it has a `dm_key`; a group must have a name. `last_seq` / `rev` are the counters bumped under the row lock.
`conversation_members`	PK `(conversation_id, user_id)`, `role ('admin'/'member')`, `last_read_seq`, `joined_at`	Index on `user_id` for "my conversations". Unread and receipts derive from `last_read_seq`.
`messages`	`id uuid` (client-generated), `conversation_id`, `seq`, `rev`, `sender_id → SET NULL`, `kind ('user'/'system')`, `text`, `image_url`, `image_width/height`, `reply_to_id → SET NULL`, `edited_at`, `deleted_at`, `topic_id → SET NULL`, `topic_source`, `topic_score`, `tsv` (generated)	`UNIQUE (conversation_id, seq)`. Index `(conversation_id, rev)` for catch-up and `(conversation_id, topic_id, seq)` for lanes. GIN on `tsv`. `topic_source` is `auto`, `reply`, `lane`, `user` or `llm`.
`reactions`	PK `(message_id, user_id, emoji)`	One of each emoji per person.
`uploads`	`id`, `user_id`, `url`, `width`, `height`, `bytes`	A message can only attach an upload its sender owns.
`group_invites`	`id`, `conversation_id`, `token UNIQUE`, `created_by`, `expires_at`, `revoked_at`, `uses`	Revocable, expiring, counted.
`topics`	`id`, `conversation_id`, `title`, `title_source ('auto'/'user'/'llm')`, `recap`, `status ('open'/'decided')`, `outcome`, `status_source`, `digest_through_seq`, `digest_at`, `digest_model`	A member's title or status is never overwritten automatically. Size, activity and unread are derived, not stored.
`message_embeddings`	`message_id PK`, `conversation_id`, `embedding vector(384)`, `model`	HNSW index with cosine ops. Records which model made each vector so a model change can be backfilled.

Table	Key columns	Why it looks like this
`topic_corrections`	`id bigserial`, `conversation_id`, `user_id`, `kind ('move'/'merge'/'rename'/'status')`, `message_id`, `from_topic_id`, `to_topic_id`, `details jsonb`	Every human correction, with what the assigner had decided: future training data.
`topic_mutes`	PK `(user_id, topic_id)`	Personal.
`message_mentions`	PK `(message_id, user_id)`, index on `user_id`	Resolved at write time, members only.
`topic_questions`	`id`, `conversation_id`, `topic_id`, `message_id`, `asked_of`, `question`, `answered`, `dismissed_at`; `UNIQUE (message_id, asked_of)`	Partial index `WHERE NOT answered AND dismissed_at IS NULL` serves the inbox.
`push_subscriptions`	`id`, `user_id`, `session_id → sessions CASCADE`, `endpoint UNIQUE`, `p256dh`, `auth`	Tied to the login that created it, so logging out stops notifications to that browser.

Deletion behaviour to know. Deleting a user cascades their sessions, memberships, reactions, mentions and push subscriptions; their messages stay with `sender_id = NULL` and show "Former member". Deleting a conversation cascades everything in it. A deleted topic leaves its messages with `topic_id = NULL` rather than deleting them.

Why UUIDs everywhere? They can be generated on the client (needed for idempotent sends), they do not reveal how many rows exist, and they cannot be guessed by incrementing. The cost is a bigger index than a bigint.

4.16 API and socket reference

REST, all under `/api`, JSON, cookie auth (everything except signup, login, health and config requires a session):

Area	Endpoints
Auth	`POST /auth/signup`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`, `PATCH /auth/profile`, `POST /auth/avatar`, `PATCH /auth/smart`, `POST /auth/invite-code`
Conversations	`GET /conversations`, `POST /conversations/dm`, `POST /conversations/group`, `GET /conversations/:id`, `GET /:id/messages?before&after&limit&topicId`, `GET /:id/changes?since`, `POST /:id/messages`, `GET /:id/topics`, `POST /:id/read`
Group admin	`PATCH /:id` (rename), `POST /:id/avatar`, `POST /:id/members`, `DELETE /:id/members/:userId`, `PATCH /:id/members/:userId` (role), `PATCH /:id/ai`, `GET/POST /:id/invites`, `DELETE /:id/invites/:inviteId`
Messages	`PATCH /messages/:id`, `DELETE /messages/:id`, `PUT/DELETE /messages/:id/reactions`, `POST /messages/:id/topic` (move)
Topics	`PATCH /topics/:id` (rename), `PUT /topics/:id/mute`, `POST /topics/:id/merge`, plus digest routes
Other	`GET /invites/:token`, `POST /invites/:token/accept`, `POST /uploads/image`, `GET /search`, `/inbox`, `/push/subscribe`, `/push/unsubscribe`, `GET /health`, `GET /config`

Errors always have the shape `{ error: { code, message, details? } }`, and validation errors include `fieldErrors` so the form can show the message next to the right field.

Socket.IO events, server to client: `message:new`, `message:updated`, `lanes:update`, `conversation:updated`, `conversation:removed`, `read`, `typing`, `presence`, `topic:mute`, `me:updated`, `inbox:update`. Client to server: `typing`, `presence:sync` (with ack), `looking`.

Why do sends go over HTTP and not the socket? HTTP gives a response with a status code, works with the same auth and rate-limit middleware, and is easy to retry. The socket is only for the server telling clients what happened. If the socket drops, nothing is lost: the HTTP send still worked, and the catch-up endpoint fills the gap.

4.17 Topic lanes in the UI: the three views

The chat pane of a group has a view per conversation, stored in the chat store: `{ mode: "chat" }`, `{ mode: "topics" }` or `{ mode: "lane", topicId }` (ChatPane.tsx).

- **Chat (mixed) view.** Every message in order, as in any messenger, each with a small coloured topic chip. A message whose topic is not decided yet shows a pending chip that fills in when `lanes:update` arrives. Clicking a chip opens that lane.
- **Topics view.** TopicsPanel.tsx lists every lane, most recently active first: colour bar, title, last message, up to three participant avatars, message count, unread badge (greyed if muted), and the AI catch-up and decided status when enabled. Buttons to mute, rename and merge.
- **Lane view (the filter).** Opening a lane loads `GET /conversations/:id/messages?topicId=<id>`, which is the same history query with `AND m.topic_id = $5`, served by the `(conversation_id, topic_id, seq)` index. It pages backwards like the main list. Pending sends are filtered to the ones aimed at this lane. The composer inside a lane sends with `topicId`, which pins the message to it (`topic_source = 'lane'`, no scoring).
- **Keeping the filtered view live.** Every `message:new` / `message:updated` goes through `applyMessage`, which also checks the open lane: if the message's `topicId` matches, it is upserted into the lane; if it was in the lane and has moved away, it is removed. So a move made by someone else is reflected immediately.
- **Colours.** `topicHue` hashes the topic id with FNV-1a into one of ten well-separated hues, so a topic has the same colour on every screen without storing a colour.
- **Corrections.** "Move to topic…" on a message, and rename/merge on a topic, call the API, which applies the change for everyone, bumps `rev`, logs a `topic_corrections` row and emits `lanes:update`.
- Lanes are only for groups, and a member can switch them off for themselves in settings; the client then always shows the chat view.

4.18 General questions they may ask about the stack

Why TypeScript? Shared shapes between routes, services and DTOs catch whole classes of bugs at compile time, and zod schemas give runtime validation of untrusted input with the TypeScript type inferred from the same schema.

Why Express 5? Mature, minimal, and Express 5 forwards rejected promises from async handlers to the error middleware, so handlers can just `throw` an `HttpError`.

Why zod? Every request body, query and params object is parsed with a schema; a failure becomes a 400 with field errors. The environment is validated the same way at startup, so a missing `DATABASE_URL` fails loudly instead of at the first query.

What does helmet do here? Security headers: a Content-Security-Policy in production (scripts only from self, images from self, data:, blob: and Cloudinary, sockets over ws/wss), `Cross-Origin-Resource-Policy: same-site`, no `X-Powered-By`, and the usual `nosniff` and frame protections.

REST vs GraphQL? The client's needs are a handful of fixed shapes, and caching and rate limiting are simpler per endpoint. GraphQL would add a layer without solving a problem I had.

How would you explain the event loop / why Node for this? Chat is I/O bound: thousands of mostly idle connections waiting on the network and the database. Node handles that well on one thread. The one CPU-heavy part, embeddings, is moved to a worker thread so it does not block the loop.

How do you handle errors? Services throw typed `HttpError`s (`notFound`, `forbidden`, `conflict`, `badRequest`). One error middleware turns them into the JSON error shape; anything unexpected becomes a 500 with the details logged, not sent.

Accessibility? Buttons have `aria-label`s, toggles use `aria-pressed`, alerts use `role="alert"`, and the mention suggestions are a labelled list. Not tested with a screen reader; say so.

---

5. Behavioural questions and stories

Each story follows situation, action, result, and what you took from it. Keep each under two minutes when spoken.

"Tell me about a hard bug."

The embedding worker that only failed in production.

- **Situation.** Everything passed in development and in the test suite. When I ran the production build for the final walk-through, the embedding model failed to load.
- **Why it was easy to miss.** The app degrades on purpose when the model is unavailable: lanes fall back to conversational signals and search falls back to keywords. So nothing crashed. The results were just worse, with one line in the log.
- **Cause.** The embedding model runs in a worker thread. A worker normally needs its own file, but the path and loader differ between `tsx` in development, Vitest, and the compiled build, so I kept the worker's code as a string and started it with `eval: true`. Code run from a string has no file of its own, so it looks for the library relative to the working directory. That only works when the server happens to be started from the right folder.
- **Fix.** The main thread resolves the library's location with `import.meta.resolve` and passes it to the worker as data. (embedder.ts)
- **Result.** The sixth walk-through now runs against the production build, so this class of bug is caught.
- **Lesson.** Test the artefact you ship. And a graceful fallback needs an alarm, or it hides the failure it was built to survive.

Before telling this one, reproduce it so you can describe exactly what you saw: start the built server from a different directory with the `import.meta.resolve` line reverted.

"What is one issue you faced, and how did you solve it?"

Similarity scores that meant different things in different languages.

- **Situation.** About a third of my test chats mix Hindi and English, and one set of weights has to serve both.
- **Investigation.** I wrote a probe that measured the similarity between pairs of messages (eval/embedding-probe.ts). Any two romanised Hindi messages scored 0.3 to 0.9, whatever they were about. English pairs scored 0.1 to 0.3. The model was detecting "these are both Hinglish", not "these are about the same thing".
- **Fix.** Subtract the conversation's mean vector from both messages before comparing, which removes what all its messages have in common. I shrink the mean toward zero for young conversations, and compute it in SQL.
- **Result.** Unrelated messages score about 0 in any language, so one set of weights serves both.
- **Lesson.** Look at the raw numbers before tuning on top of them. I would have spent days adjusting weights around a signal that was measuring the wrong thing.

"Tell me about a time something you built did not work."

Three attempts to fix compounding errors, all removed.

- **Situation.** The assigner made the right call 87% of the time given the true earlier lanes, but under 60% of messages ended up in the right lane when replayed, because early mistakes compound.
- **What I tried.** Beam search, keeping several candidate groupings alive: accuracy fell to 45%, because the model rates "everything in one lane" as most probable. A small language model to score "does this message follow that lane": it picked the right lane 54% of the time, against 81% for simply taking the lane that spoke last. Five interaction features aimed at catalogued mistakes: no change.
- **Result.** I removed all three and wrote down what was tried and the numbers.
- **Lesson.** A negative result is still a result if it is measured and recorded. The record stops me, or anyone else, trying the same thing again. And I did not leave dead code in to look busy.

"Tell me about a time data changed your mind."

The LLM did worse when I gave it more information.

- **Situation.** The first tidy-up design showed the model the local assigner's lane for each message and asked it to correct the mistakes. That seemed obviously helpful.
- **Data.** 74.4% held out. The same model sorting from the text alone, with no guess shown: 85%.
- **Action.** Rebuilt the tidy-up to hide the guess for messages that had not yet been sorted, then ran it once on each set with no further prompt changes.
- **Result.** 86.5% held out.
- **Lesson, and the honest part.** That means the held-out set was used once to choose a design, so 86.5% is a held-out figure for the prompt but not for that one decision. I say so in the write-up.

"Tell me about a trade-off you made."

Pick one:

- **Instant and roughly right over slow and right.** A message gets a lane within a fraction of a second at about 58% accuracy, and is corrected a minute or two later at about 86% where AI is on. Waiting for the better answer would make the feature feel broken.
- **Over-splitting over mixing.** The assigner produces 19 lanes for 15 true topics. I leaned that way on purpose: merging two lanes is one action, while untangling a mixed lane is many. That is why the UI makes merging easy.
- **Privacy over accuracy by default.** The LLM gives 86% against 58%, and it is still off by default, because turning it on sends a group's messages to a third party. That is the admin's decision to make.
- **No end-to-end encryption.** The core feature needs the text on the server.

"Tell me about a time you had to work within a constraint."

A free-tier API quota.

- **Situation.** The only LLM key I had was a free-tier Gemini key. An evaluation run stalled for a long time retrying 429 and 503 responses before I found the cause: a hard limit of 20 requests a day on the larger model. One evaluation run needs about 22 requests for the held-out chats and 72 for the tuning chats.
- **Action.** Switched to the lite model, which only has per-minute limits. Counted the requests a run needs before starting it. Cached every answer on disk, so re-running the script reproduces the same figures without spending quota.
- **Result.** The published numbers are reproducible, and the documentation says which model they were measured with and that other models are unmeasured.

"What are you most proud of?"

Two options. The second is the stronger answer for most interviewers.

- The messaging protocol: one `rev` counter gives ordering, live updates and reconnect catch-up for every kind of change, including ones added later like topic moves.
- The evaluation write-up. It says the product's weights are no better than the baseline on IRC, that VI shows no gain, and that the test data is small and mine. I would rather show a 58% I can defend than a 90% I cannot.

"What would you do differently?"

- **Start with real data.** I wrote the test chats myself. I would recruit two or three real groups early and have two people label them.
- **Protect the test set.** I scored the held-out chats in aggregate while iterating. I would lock them away and score once.
- **Let the assigner revise.** It commits each message once. Letting it revise its last few decisions when a new topic clearly begins is the most promising fix for missed topic starts.

"How did you decide what to build first?"

Phase by phase, each ending with a walk-through script: foundation and reliable messaging first, then the rest of the chat features, then lanes and the evaluation, then mute and search, then the LLM features, then the PWA and push. The smart features all depend on messaging being correct, so ordering, idempotency and catch-up came before any of them.

"Tell me about a time you found a bug in your own design."

Settings switches that lost updates. Flipping several feature switches quickly could lose one. There were two halves to the fix. On the server, the change is now applied to the stored JSON in a single `UPDATE` statement, so two switches flipped at the same moment cannot overwrite each other. In the client, each response carries the whole settings object, and the answer to an earlier request does not yet include the later flips, so the client counts requests in flight and only applies the answer to the last one. (auth/routes.ts, useSmartStore.ts)

---

6. The awkward questions

These are the ones that sink people who are not ready for them. Answer directly and briefly, then stop.

"58% is not very good, is it?" No, and I do not claim it is. It is 18 points above doing nothing, on a problem that is hard enough to be a research benchmark. Topic starts are the weak point: it missed 8 of 15. I designed around that: any member can correct a lane, the correction applies for everyone, and each one is logged as training data.

"You wrote the test data and the model. Is that not marking your own homework?" Yes, and the write-up says so. That is why I also ran it on the public IRC benchmark with the published metrics, and reproduced the paper's baseline exactly to check my metric code. The casual numbers are indicative, not proof.

"On IRC you beat the paper. Is that a fair claim?" I would not put it that way. With weights fitted on the IRC dev split, the same code matches the paper's feedforward model on VI and is ahead on one-to-one, and it is behind on exact-match F1. Those are not the product's weights, I fitted on 2,500 messages instead of the full training split, and it is one run. The claim I would make is that the feature design is sound enough to be competitive on the standard benchmark.

"Did you use AI tools to build this?" The current history has no AI co-author trailers, but the question comes up for any polished solo project. Answer honestly and the same way every time: say which tools you used and for what, and that you made the design decisions, ran the experiments and read what went in. Then prove it by explaining any part in detail; this document exists so that you can. An interviewer cares far less about who typed a line than about whether you can say why it is there, what the alternatives were, and what breaks if you change it.

"The whole history is one commit." The public repository has a single "Initial commit". Be ready to say how long the work really took and how you worked (phase by phase, each ending in a browser walk-through, see section 5). If asked why there are no small commits, say the history was squashed before publishing and that on a team you would use feature branches and pull requests with reviewable commits.

"This started as a tutorial project?" Yes. The starting point was a MERN tutorial chat app. I kept the frontend stack (React, Zustand, Tailwind with daisyUI, Socket.IO) and replaced nearly everything else: MongoDB with Postgres, JavaScript with TypeScript, and the authentication, socket handling and data model. The technical notes list eight security problems in the original and what replaced each one.

"Has anyone used it?" Say what is true. If the answer is only you and the demo data, say so, and say the next step is real groups, because that is also what the evaluation needs.

"Why not just use an LLM for everything?" Cost per message, latency, and privacy. The local assigner costs nothing per message, answers in milliseconds, and sends nothing off the server. The LLM is used where it is strong, re-sorting a whole exchange at once, and only where a group has agreed to it.

"Why not train a neural model?" With 600 labelled messages I would be fitting noise. A linear model over hand-built features is the right size for that much data, and it is competitive on the benchmark. With the correction log filling up from real use, a small learned model of "does B continue A" becomes worth trying.

"The README links to a live demo that does not work." Fix this before any interview. See section 9.

"What part of this do you understand least?" Have an honest answer ready, and pick something you have since read up on. A reasonable one: the internals of HNSW, or why the 1.5 boost on content features helps on replay but not in the likelihood.

---

7. System design follow-ups

Interviewers often turn a project discussion into "now scale it".

"How would you run several instances?" Nothing has been built for it, but three things would change, and the design does not block any of them:

1. **Socket.IO.** Events are addressed to per-user rooms and membership is read at emit time, so adding the Redis adapter is a small change.
2. **The job queue.** It is an interface with two methods. A Redis-backed queue with one group per conversation, such as BullMQ, fits the same shape. Jobs are already safe to run twice.
3. **Process-local state.** Presence counts and rate-limit counters move to the shared store.

The database guarantees already hold across instances, because ordering comes from a row lock and not from anything in process memory.

"What breaks first at 100,000 users?" In order:

1. The embedding worker. One thread at 3 ms per message is about 300 messages a second at best. Fix: a pool of workers, or a separate embedding service, with batching.
2. Emitting to a group reads its member list from the database every time. Fix: cache membership with invalidation on change.
3. The HNSW index is global across all conversations, and filtered search gets slower as the share of rows a caller can see shrinks. Fix: partition embeddings, or pre-filter by conversation.
4. The conversation row lock, for very large groups.

"What if a group has 10,000 members?" The design assumes small groups. Fan-out by emitting to 10,000 user rooms per message, a `read` event per reader and a serialised row lock would all need rethinking: per-conversation rooms, batched or sampled read receipts, and a sequence allocator in place of the lock. The assigner's "crowd" feature exists because public channels behave differently, but the shipped weights are fitted to small groups.

"How would you add message history export, or deletion for compliance?" Deleting a message already erases its content and removes its embedding and mentions. There is no account deletion endpoint yet, but the schema is ready for one: sessions, memberships and push subscriptions cascade, and messages keep a null sender. A retention policy and an export endpoint are listed as not built.

"How would you change the embedding model?" The vector column is fixed at 384 dimensions, and each row records which model produced it. Changing models means adding a second column or table, backfilling in the background with the existing queue, and switching reads over when the backfill is done.

"How would you make the LLM step cheaper at scale?" It already runs once per quiet spell and not per message, sends at most 80 messages cut at 500 characters, and never runs with nothing new. Next: skip the tidy-up when the local assigner's scores were all confident, and batch several groups' requests where the provider's batch pricing allows.

"How would you monitor it?" Today there are console logs only. I would add: assignment latency from send to `lanes:update`, queue depth per conversation, embedding failures, LLM error and refusal rates, and the correction rate per group, which is the best live measure of lane quality.

---

8. Concepts to explain from scratch

Be able to explain each of these in two or three sentences without notes. An interviewer who suspects you are reciting will ask for a definition.

Concept	Plain explanation
Idempotency	Doing an operation twice has the same effect as doing it once. A send with a client-chosen id is idempotent: the second attempt finds the first one's row.
`SELECT ... FOR UPDATE`	Locks the selected rows until the transaction ends. Any other transaction that tries to lock or update them waits. Used here to serialise all writes to one conversation.
Optimistic UI	Show the result of an action before the server confirms it, and reconcile when the response arrives.
Embedding	A list of numbers (384 here) that represents a text, produced by a model so that texts with similar meaning have vectors that point in similar directions.
Cosine similarity	The cosine of the angle between two vectors: 1 for the same direction, 0 for unrelated, negative for opposite. It ignores vector length.
HNSW	Hierarchical navigable small world: a graph index for approximate nearest-neighbour search. It is much faster than comparing against every vector, at the cost of occasionally missing a true neighbour.

Concept	Plain explanation
GIN index	Generalised inverted index: maps each word to the rows containing it, which is what makes full-text search fast.
Reciprocal rank fusion	Combine several ranked lists by giving each item 1 / (k + rank) per list and adding. Needs only ranks, not comparable scores.
Softmax	Turns a list of scores into probabilities that sum to 1, with higher scores getting exponentially more of the mass.
Maximum likelihood	Choose the weights that make the observed correct answers most probable under the model.
L2 regularisation	A penalty on large weights added to the objective, which stops the model fitting noise in a small dataset.
Leave-one-out cross-validation	Fit on all chats but one, test on the one left out, repeat for each chat, and average. Used here per chat, not per message, so no chat is in both sets.
Held-out set	Data never used for fitting or tuning, kept to estimate performance on new data.
Variation of information	A distance between two clusterings from information theory: how much you would still need to be told about one if you knew the other.
Conversation disentanglement	The research name for the problem: separating interleaved conversations in one message stream.
Worker thread	A second JavaScript thread in the same Node process. CPU-heavy work there does not block the event loop.
Event loop	Node runs JavaScript on one thread, taking one callback at a time. A long synchronous computation blocks everything else.
CSRF	Cross-site request forgery: another site makes the user's browser send a request with their cookies. Prevented with SameSite cookies and an Origin check.
SameSite=Lax	The browser sends the cookie on same-site requests and on top-level navigations, but not on cross-site POSTs or embedded requests.
httpOnly cookie	Not readable from JavaScript, so a script injection cannot steal it.
VAPID	A key pair that identifies the application server to browser push services, so only this server can push to its subscriptions.
Service worker	A script the browser runs in the background for a site. It can intercept network requests, cache responses and receive push messages while no tab is open.
Prompt injection	Text in the model's input that tries to act as instructions. Defended here by escaping, by having no tools, and by validating the output against what was sent.
Structured output	Constraining a model's answer to a JSON schema, so it can be parsed and validated instead of scraped from prose.
Tombstone	A row kept after deletion with its content erased, so that ordering and references to it still work.

9. Before the interview

Fix in the repository

- [x] Live demo link is real now: synchat.arvin.life. Open it the day before: the free Render plan sleeps, so wake it up and check the demo logins (`asha`, `ben`, `chidi`, `dev` / `synchat-demo-2026`) still work and the demo data has not been vandalised by visitors. Re-seed if it has.
- [x] Clone URL points at `github.com/Arvin-Arun-Dcosta/SynChat`.
- [ ] The README still quotes "58% (41% for doing nothing)". The 58% is the held-out figure, whose baseline is 40.1%; 41.6% is the baseline of the cross-validated set, which scores 56.7%. Quote a matching pair (58% vs 40%).
- [x] `interview.md` is in `.gitignore` (it is personal prep and stays local).
- [ ] The git history is a single "Initial commit". Be ready for "why one commit?" (see section 6).

Run once, so you are quoting what you saw

- [ ] `npm test` and note the number of tests it reports.
- [ ] `npm run eval -- --baselines` in `backend/`, to see the accuracy table print.
- [ ] `./run.sh seed`, then open the Flatmates group, the Topics view and search.

Rehearse a five-minute demo

1. Open the seeded Flatmates group and point at the topic chips.
2. Switch to the Topics view.
3. In a second browser as another person, send two interleaved messages and watch the chips appear.
4. Move a message to another lane and show it change on the other screen.
5. Mute a topic, then `@mention` yourself from the other browser.
6. Search "where are we going to eat this weekend" and show a match with no shared words.

Read these five files until you can explain every line

1. backend/src/messages/service.ts: `sendMessage`, `bump`, `listChanges`
2. backend/src/lanes/assigner.ts: `featurise`, `assign`
3. backend/src/lanes/service.ts: `process`, `assignInTx`, `recentWithSimilarity`
4. backend/src/digest/tidy.ts: `planTidy`
5. backend/src/search/service.ts: `search`

Questions to ask them

- How do you evaluate a feature whose output is a judgement call, where there is no single right answer?
- Where do you run inference today: your own hardware, or a provider's API? What drove that?
- How does the team decide when an offline metric is good enough to ship?